C++ function overloading

```
#include<iostream>
using namespace std;
const float PI = 3.14;
double area(double length)            // Area of square
{
        return length * length;
}
double area(double length, double width)     // Area of rectangle
{
        return length * width;
}
double area(int radius)      //Area of circle
{
        return PI * radius * radius;
}
int main()
{
        cout << area(2) << endl;      // 12.56
        cout << area(2.0) << endl;    // 4
        cout << area(1.1, 2) << endl; // 2.2
        return 0; }
```

C++ function overloading

- Function overloading allows us to use same function name for multiple function definitions
  - Only related functions should be overloaed
- All the overloads of a function must have unique signature
  - Signature of the function includes **number, type** and **order** of parameters
    - Functions with same name but different number of arguments are valid overloads
    - Functions with same name and same number of arguments, but different types are valid overloads
      - int area(int x, float y);
      - Int area(int x, double y);
    - Functions with same name and same number and type of arguments, but different order of arguments are valid overloads
      - int area(float y, int x);
      - Int area(int x, float y);
  - Signature of the function does not include return type
    - Followig two are not valid overloads, it will result in error
      - int area(int x, float y);
      - float area(int x, float y);

C++ function overloading

- Function selection (a.k.a overload resolution) is done by compiler during compilation
  - For each function call compiler decides which overload of the function to call based on actual arguments passed during function call
- It follows following rules for each function call:
  1. Prepare list of **candidate functions** (All overloads with same function name)
  2. Select **viable functions** from candidate functions (based on # of arguments and feasibility of conversion of arg type to parameter type e.g. int to char * is not feasible)
  3. One function from viable functions is called based on following rules
     I. Exact match
     II. Promotions
        - integral promotions - bool, char, short, enum to int
        - floating-point promotions - float to double
     III. Standard type conversions
        - e.g. double to float, int to long, int to float, bool to short, bool to float etc.
     IV. User-defined conversions (related to classes, ignore for now)

C++ function overloading

I. Exact match
II. Promotions
    - integral promotions - bool, char, short, enum to int
    - floating-point promotions - float to double
III. Standard type conversions
    - e.g. double to float, int to long, int to float, bool to short, bool to float etc.
IV. User-defined conversions (related to classes, ignore for now)

- **IMPORTANT: There is an overall best match if there is one and only one function for which, match for each argument is no worse than match required by any other viable function**

C++ function overloading

A) double area(int r);
B) double area(double r);

Sr. No.	Function call	Viable functions	Exact match	Prom-otions	Std. type conversions	Remark
1	area(10);	A, B	A			Calls A
2	Area(10.0f);	A, B	-	B		Calls B
3	area(true);	A, B	-	A		Calls A
4	area('A');	A, B	-	A		Calls A
5	Area(10.0);	A, B	B			Calls B
6	area(short(10));	A, B	-	A		Calls A
7	area(10l)	A, B	-	-	A, B	Ambiguous call
8	int i; area(&i);	-	-	-	-	No matching f^n

C++ function overloading

```
void fun(int x, int y) {
    cout << "fun1" << endl;
}

void fun(int x, long y) {
    cout << "fun2" << endl;
}

int main() {

    fun(true, 5);
    return 0;
}
```

C++ function overloading

```
void fun(int x, double y) {
    cout << "fun1";
}

void fun(bool x, float y) {
    cout << "fun2";
}

int main() {

    fun(true, 5.1);
    return 0;
}
```

C++ function overloading

```
void fun(double x) {
    cout << "fun1" << endl;
}

void fun(long x) {
    cout << "fun2" << endl;
}

int main() {

    fun(5);
    return 0;
}
```

C++ function overloading

```
void fun(double x) {
    cout << "fun1" << endl;
}

void fun(long x) {
    cout << "fun2" << endl;
}

int main() {

    fun(5.1);
    return 0;
}
```

C++ function overloading

```
void fun(long double x) {
    cout << "fun1" << endl;
}

void fun(long x) {
    cout << "fun2" << endl;
}

int main() {

    fun(5.1);
    return 0;
}
```

C++ function overloading

```
#include<iostream>
#define PI 3.14
using namespace std;
double area(int r)
{
	cout << "A" << endl;
	return PI * r * r;
}

double area(int l, double w = 10.0)
{
	cout << "B" << endl;
	return l * w;
}
```

```
int main()
{
	cout << area(10, 10.0) << endl;
	//cout << area(10) << endl;

	cout << area(true, 10.0) << endl;
	//cout << area('A') << endl;

	cout << area(true, 10) << endl;
	//cout << area(10.0) << endl;
	//int i; cout << area(&i) << endl;

	return 0;
}
```

C++ function overloading

```
void fun(float f, int i) {
    cout << "fun1" << endl;
}
void fun(int i, int j) {
    cout << "fun2" << endl;
}
void fun(int i, float f) {
    cout << "fun3" << endl;
}

int main() {
    fun(true, true);
    return 0;
}
```

C++ function overloading

```
void fun(float f, int i) {
    cout << "fun1" << endl;
}
void fun(int i, int j) {
    cout << "fun2" << endl;
}
void fun(int i, float f) {
    cout << "fun3" << endl;
}

int main() {
    fun(true, true);
    return 0;
}
```

1. **`fun(float, int)`**
    1. First `bool → float` ✅ (standard conversion)
    2. Second `bool → int` ✅ (standard conversion)
2. **`fun(int, int)`**
    1. Both `bool → int` ✅ (standard conversion)
3. **`fun(int, float)`**
    1. First `bool → int` ✅
    2. Second `bool → float` ✅

So **all three overloads are viable**.

C++ function overloading

<u>Best match ranking</u>

C++ ranks conversions:

- Exact match is better than promotion.
- Promotion is better than general conversion.

Now check each parameter:

- For `fun(int, int)` → both parameters are `bool → int` (**promotion**, since `bool` promotes directly to `int`).
- For `fun(float, int)` → first is `bool → float` (**conversion**, not promotion), second is `bool → int` (promotion).
- For `fun(int, float)` → first is `bool → int` (promotion), second is `bool → float` (conversion).

Clearly, **`fun(int, int)` is the best match**, because both parameters are promotions (better than conversion).

From → To	Category	Notes
bool → int	**Promotion**	bool promotes directly to int (value: false=0, true=1)
char → int	**Promotion**	char, signed char, unsigned char, short → promoted to int (or unsigned int)
wchar_t, char8_t, char16_t, char32_t → int	**Promotion**	Promotes to an int type large enough to hold values
float → double	**Promotion**	Always safe, no precision loss expected
int → float	**Conversion**	May lose precision if int too large
int → double	**Conversion**	Safer, but still a conversion, not promotion
float → int	**Conversion**	Fractional part truncated
double → int	**Conversion**	Loss of fraction + possible overflow
unsigned int → float/double	**Conversion**	May lose precision if values exceed mantissa
enum → int	**Promotion**	Underlying type promoted to int
pointer → bool	**Conversion**	nullptr → false, non-null → true

When mixing types in arithmetic, C++ applies rules to get a **common type**:

Example	**Result Type**
int + float	float
int + double	double
float + double	double
unsigned int + int	Depends on rank; usually promoted to unsigned int if same size
short + int	int (promotion)

Modern C++ features

```
auto sum(int x, int y) {
    return x + y;
}

int main() {
    cout << sum(1, 2) << endl;
    return 0;
}
```

- When you use auto in a function's return type, the compiler **deduces ONE common type** for the return expression.
- But **all return statements must resolve to the same type**, otherwise the compiler gives an error.

Modern C++ features

```
auto fun(int x, int y) {
    if(x > 0)
        return x + y;
    else
        // error: inconsistent deduction for auto return type: ‘int’ and then ‘double’
        return x * 2.0;
}

int main() {
    cout << fun(1, 2) << endl;
    cout << fun(-1, 2) << endl;
    return 0;
}
```

Modern C++ features

```
auto fun(int x, int y) {
    if(x > 0)
        return x + double(y);
    else

        return x * 2.0;
}

int main() {
    cout << fun(1, 2) << endl;
    cout << fun(-1, 2) << endl;
    return 0;
}
```